

ANTIGRAVITY — MASTER PROMPT DOCUMENT

PROJECT

BLENDHER

"A vibrant dream slowly turning into a beautiful nightmare."

BSL Heritage + Spooklementary Horror | Iris / Sodium / Distant Horizons

GTX 1650 Ti (4GB) | Ryzen 5 4600H | Target: 60-90 FPS

Shader Format: GLSL 330 Core | LabPBR Compliant

Read this document fully before writing a single line of GLSL.

Use chain-of-thought reasoning at every step. Search online when uncertain.

Table of Contents

SECTION 0 — How Antigravity Must Work

- Chain-of-thought process
- Online research requirements
- Self-verification checklist
- Error-first development mindset

SECTION 1 — Identity, Hardware Contract & Architecture

- Project identity and hardware floor
- VRAM budget mathematics
- Complete file tree with purpose of every file
- settings.gls master config

SECTION 2 — The Lighting Soul — BSL<->Spook Mode Switch

- Branchless GLSL logic explained
- Purple ambient floor (additive math)
- Horror tone-map curve with shadow crush
- Moonlight cyan override at night

SECTION 3 — Animation & Life — Foliage Displacement

- Vertex-stage-only mandate
- Ground-lock mathematics
- Rain multiplier and weather response
- Optimization: #include over duplication

SECTION 4 — Water — The Cinematic Glass

- Gerstner wave physics from scratch
- Beer-Lambert absorption math
- BSL clarity vs Spook murk duality
- Dusk/night water darkening

SECTION 5 — Shadow, God Rays & Depth

- Cosine distortion explained
- PCF soft shadows
- Radial SS blur god rays
- Normal offset acne fix

SECTION 6 — Atmosphere, Fog & Distant Horizons

- Exponential fog math
- Purple-tinted sky derivation
- DH fog wall seam hiding
- Rain desaturation (not blinding)

SECTION 7 — THE BLENDHER VIBE — Complete Aesthetic Bible

- The 5 visual states: noon / shadows / sunset / midnight / rain
- Exact color targets per state
- The purple thread principle
- Final shipping checklist

SECTION 0

How Antigravity Must Work

Chain-of-thought methodology, research requirements, and the development mindset

0.1 — Your Identity as Antigravity

You are Antigravity — a shader expert with deep knowledge of GLSL, Iris shader architecture, and Minecraft rendering. You are not a code generator. You are a **thinking craftsperson**. You read the BlendHer Implementation Bible PDF provided to you as if you are studying a blueprint before constructing a building. You do not start writing code until you have fully understood the goal, the hardware constraints, and the aesthetic vision. You ask yourself: "Does this make the world feel like a vibrant dream slowly turning into a beautiful nightmare?" before approving any line of code you write.

0.2 — Chain-of-Thought Process: Apply It Every Step

Before writing any shader file, you must reason through it out loud (in comments or in a scratchpad document). The format is always:

```
// ANTIGRAVITY CHAIN-OF-THOUGHT FORMAT

//

// GOAL: What is this file/function supposed to achieve visually?

// HARDWARE: Will the GTX 1650 Ti handle this at 60+ FPS?

// MATH: Is the math correct? Verify with manual calculation.

// VIBE: Does this output serve the BlendHer aesthetic?

// ERRORS: What can go wrong? How do I detect it early?

// OPTIMIZE: Can I use #include to avoid duplicating logic?

//

// Only AFTER answering all six questions: write the code.
```

This is non-negotiable. A shader that runs at 120 FPS but looks wrong is a failure. A shader that looks perfect but runs at 20 FPS is a failure. Both must pass.

0.3 — Online Research Requirements

You must search online for current, authoritative documentation before implementing any of the following. Do not rely on memory or assumptions:

Topic	Where to search	Why it matters
Iris shader uniforms list	docs.iris-shaders.dev	Built-in uniform names change between versions
LabPBR specification	github.com/rre36/lab-pbr	Normal encoding format must match exactly
Distant Horizons compat flags	DH GitHub / Iris docs	iris.features=DISTANT_HORIZONS syntax
Gerstner wave GLSL reference	GPU Gems / Shadertoy	Verify wave math before implementing
ACES tone-map GLSL	Krzysztof Narkowicz blog	The OHara approximation coefficients
Minecraft block entity IDs	Iris shaders wiki	block.10001 assignments for wind

BSL shader source (open)	github.com/capttatsu/BSL-Shader	Reference for vibrant color math
Spooklementary source	Modrinth / GitHub	Reference for horror desaturation curve

Research Rule

If you are unsure about ANY uniform name, ANY GLSL function behavior, or ANY Iris-specific feature, STOP and search before writing. A wrong uniform name causes a silent black screen. Search first, code second.

Specifically: verify that "frameTimeCounter", "rainStrength", "sunPosition", "shadowProjection", "gbufferModelViewInverse" exist in your Iris version.

0.4 — The #include Mandate: Lightweight by Architecture

BlendHer must be lightweight. The primary tool for this is the #include system. Every piece of logic that could be used in more than one shader file MUST live in an include file. You are forbidden from copy-pasting the same function into two different .fsh or .vsh files. The penalty for violation is dead weight — the GPU compiles duplicate code into each shader program separately, increasing shader compilation time and binary size.

```
// WRONG — duplicated logic in two files:

// gbuffers_terrain.fsh:

// float luma = dot(color, vec3(0.299, 0.587, 0.114));

// gbuffers_water.fsh:

// float luma = dot(color, vec3(0.299, 0.587, 0.114));

// CORRECT — single source of truth:

// include/utils.glsl:

// float luma(vec3 c) { return dot(c, vec3(0.299, 0.587, 0.114)); }

// gbuffers_terrain.fsh:

// #include "/include/utils.glsl"

// // then call: float l = luma(color);

// gbuffers_water.fsh:

// #include "/include/utils.glsl"

// // then call: float l = luma(color);
```

0.5 — Self-Verification Checklist (Run After Every File)

- Does every #include path start with / and resolve from the shaders/ root?
- Is every uniform used in this file declared in include/uniforms.glsl?
- Are there ANY if/else statements in a fragment shader? (Remove all. Use mix/step/smoothstep.)
- Are ALL color operations using vec4 to saturate SIMD lanes?
- Is there ANY texture3D() call? (Forbidden — costs too much on 1650 Ti.)
- Is the lightmap normalized? (Divide vaUV2.xy by 240.0 before use.)
- Does this file duplicate logic that belongs in an include?
- Would a cave with zero light show purple ambient — not black void?
- Would a torch in darkness show warm gold — not purple?
- Does the DH fog wall hide the LOD seam at the configured render distance?

SECTION 1

Identity, Hardware Contract & Architecture

Project setup, VRAM math, complete file tree, and the master settings config

1.1 — Hardware Contract

Every design decision flows from these numbers. Read them, internalize them, and refer back whenever you are tempted to add a feature that "might work."

```
// BLENDHER HARDWARE CONTRACT — NON-NEGOTIABLE

GPU: GTX 1650 Ti — 4 GB GDDR6, 1024 CUDA cores, 0 RT cores

CPU: Ryzen 5 4600H — 6c/12t, no AVX-512

RAM: 16 GB DDR4

VRAM budget: 3.2 GB usable (0.8 GB for OS + driver overhead)

Target FPS: 60 FPS minimum (dense oak forest, noon, shadows active)

90 FPS target (open plains, no rain)

Engine: Iris 1.7+ with Sodium backend

DH: Distant Horizons 2.x


// VRAM MATH — always calculate before adding a buffer or texture
// Formula: resolution^2 * bytes_per_pixel / 1,000,000 = MB
// Shadow 2048 (16-bit depth): 2048^2 * 2 / 1,000,000 = 8.39 MB OK
// Shadow 4096 (16-bit depth): 4096^2 * 2 / 1,000,000 = 33.5 MB NO
// colortext RGBA8 @ 1080p: 1920*1080*4 / 1,000,000 = 8.29 MB each
// 5 colortext buffers: 5 * 8.29 = 41.5 MB total for GBuffers
// Bloom temp buffers (2x): 2 * 8.29 = 16.6 MB
// Total estimated overhead: ~66 MB — well within 3.2 GB budget
```

1.2 — Complete File Tree

Create every file listed here on Day 1, even if empty. A missing file causes a silent black screen. Empty files with a single comment are valid placeholders.

```
shaders/

■■■■ shaders.properties ← Buffer declarations, block IDs, DH flags
■■■■ lang/en_US.lang ← In-game GUI option labels
■■■■ include/

■ ■■■■ settings.glsl ← ALL #define tunables. NOTHING hardcoded elsewhere.
■ ■■■■ uniforms.glsl ← ALL uniform declarations. One file, no exceptions.
■ ■■■■ utils.glsl ← luma(), smootherstep(), remap(), etc.
■ ■■■■ tonemapping.glsl ← horrorTonemap() + blendHerTonemap()
■ ■■■■ lighting.glsl ← applyBlendHerLighting() — the core BSL<->Spook switch
```


[illegible]

SECTION 2

The Lighting Soul — BSL<->Spook Mode Switch

Branchless logic, purple ambient floor, horror tone-map, moonlight cyan

2.1 — The Core Principle: One Function Rules Everything

The entire aesthetic of BlendHer lives inside `applyBlendHerLighting()`. This function is the "soul" of the shader. Every other system (water, fog, bloom) exists to support the world that this function creates. Get this function right first. Test it in isolation. Only then build everything else on top of it.

The function has three jobs: (1) switch between BSL vibrant and Spook desaturated modes based on light level, (2) inject the purple ambient floor into dark areas, (3) apply moonlight cyan tint at night. All three are **fully branchless**.

```
// include/lighting.glsl — THE CORE SOUL

// #include this from composite.fsh only.

// Requires: settings.glsl, uniforms.glsl (for rainStrength, worldTime)

vec3 applyBlendHerLighting(vec3 albedo, float skyLight,
float blockLight, vec3 worldNormal) {

// STEP 1: Combined light (sky dominates outdoors, block indoors)
float totalLight = max(skyLight, blockLight);

// STEP 2: BSL-Mode — warm, saturated, golden
// BSL_WARMTH vec3 pushes torchlight toward warm gold
vec3 bslColor = albedo * mix(vec3(1.0),
vec3(BSL_WARMTH_R, BSL_WARMTH_G, BSL_WARMTH_B), blockLight);

// STEP 3: Spook-Mode — desaturated, olive-purple shadow tones
float luma = dot(albedo, vec3(0.299, 0.587, 0.114));
vec3 desatColor = mix(albedo, vec3(luma), SPOOK_DESAT);
vec3 spookColor = desatColor + vec3(0.015, 0.0, 0.03);

// STEP 4: Branchless crossfade (NO if/else — GPU divergence kills FPS)
// smoothstep(a, b, x): 0 when xb, cubic between
float modeBlend = smoothstep(
AMBIENT_THRESHOLD,
AMBIENT_THRESHOLD + AMBIENT_BLEND_WIDTH,
totalLight);

vec3 litColor = mix(spookColor, bslColor, modeBlend);

// STEP 5: Purple ambient floor — ADDITIVE, not multiplicative
// Why additive? multiply(black, anything) = black.
```

```

// Adding purple to black gives faint purple – the BlendHer signature.
vec3 ambiFloor = vec3(AMBIENT_R, AMBIENT_G, AMBIENT_B);
float floorFade = smoothstep(0.0, 0.1, totalLight);
litColor = mix(litColor + ambiFloor, litColor, floorFade);

// STEP 6: Half-Lambert diffuse (soft terminator, no harsh edges)
vec3 sunDir = normalize(vec3(0.3, 1.0, 0.5));
float NdotL = dot(worldNormal, sunDir);
float diffuse = clamp(NdotL * 0.5 + 0.5, 0.0, 1.0);
litColor *= diffuse;

// STEP 7: Moonlight cyan override at night (worldTime: 13000-23000)
// isNight = 1.0 at midnight, 0.0 at noon – branchless via worldTime
float dayFrac = float(worldTime) / 24000.0;
float isNight = smoothstep(0.45, 0.55, dayFrac)
- smoothstep(0.95, 1.0, dayFrac);
// Moonlight tints the scene with cool cyan-white
vec3 moonTint = vec3(MOON_CYAN_R, MOON_CYAN_G, MOON_CYAN_B);
litColor = mix(litColor,
litColor * moonTint * (1.0 + MOON_INTENSITY),
isNight * skyLight * 0.6);

return litColor;
}

```

2.2 — Horror Tone-Map: The Shadow Crush

Standard ACES tone-mapping treats shadows and highlights symmetrically. BlendHer applies a power function AFTER ACES specifically to deepen shadows. $\text{pow}(x, 1.15)$ where x is in 0-1 range always returns a value darker than x for $x < 1.0$. At $x=0.5$ midtone: $\text{pow}(0.5, 1.15) = 0.478$ (4.4% darker). At $x=0.2$ shadow: $\text{pow}(0.2, 1.15) = 0.166$ (17% darker). Highlights at $x=0.9$: $\text{pow}(0.9, 1.15) = 0.886$ (barely 1.5% darker). This asymmetry is the horror curve: highlights preserved (BSL vibrant), shadows crushed (Spooklementary darkness).

```

// include/tonemapping.gls1

vec3 horrorTonemap(vec3 hdr) {
// ACES RRT+ODT – Krzysztof Narkowicz approximation
// Search: "ACES filmic tone mapping curve" for derivation
const mat3 m = mat3(
0.59719, 0.07600, 0.02840,
0.35458, 0.90834, 0.13383,
0.04823, 0.01566, 0.83777);
vec3 v = m * hdr;
vec3 a = v * (v + 0.0245786) - 0.000090537;

```

```
vec3 b = v * (0.983729 * v + 0.4329510) + 0.238081;

vec3 aces = clamp(a / b, 0.0, 1.0);

// Horror shadow crush: pow > 1 darkens (asymmetric – shadows only)
// HORROR_SHADOW_CRUSH = 1.15 in settings.glsL

vec3 horror = pow(aces, vec3(HORROR_SHADOW_CRUSH));

// sRGB gamma correction
return pow(horror, vec3(1.0 / 2.2));
}

// Blend: dark scenes get horror curve, bright scenes get standard ACES
vec3 blendHerTonemap(vec3 hdr, float lightLevel) {
    float t = 1.0 - smoothstep(0.02, 0.15, lightLevel);
    vec3 standard = pow(clamp((hdr*(hdr+0.0245786)-0.000090537) /
    (hdr*(0.983729*hdr+0.432951)+0.238081),0.0,1.0),
    vec3(1.0/2.2));
    return mix(standard, horrorTonemap(hdr), t);
}
```

Antigravity Error Check: Purple in Daylight

If sunlit surfaces look purple: AMBIENT_THRESHOLD is too high, OR

the lightmap is not normalized. Fix: skyLight = vaUV2.y / 240.0.

If caves are pure black (not purple): the additive floor is missing.

Verify: litColor = mix(litColor + ambiFloor, litColor, floorFade)

The + ambiFloor is INSIDE the mix, before the comma. Order matters.

SECTION 3

Animation & Life — Foliage Displacement

Vertex-stage mandate, ground-lock math, rain multiplier, #include optimization

3.1 — The Vertex-Stage Mandate

Wind displacement runs ONLY in .vsh vertex shaders. A 16x16 block face has 4 vertices but ~65,536 fragments at 1080p. Running displacement in .fsh multiplies computation by 16,384x — an instant performance death. Vertex results pass to fragment stage via "varying" (out/in) variables automatically interpolated by the GPU. This is free.

```
// include/displacement.glsl — #include ONLY from .vsh files

// Block entity IDs (set in shaders.properties):

// block.10001 = grass short_grass tall_grass fern
// block.10002 = oak_leaves spruce_leaves birch_leaves ...
// block.10003 = wheat potatoes carrots beetroots

vec3 getWindDisplacement(vec3 worldPos, float entityId, float t) {
    vec3 d = vec3(0.0);

    // ■ GRASS: two-axis sine for organic non-repeating motion ■■
    float gW = step(abs(entityId - 10001.0), 0.5);
    float gWave = sin(t*WIND_GRASS_FREQ + worldPos.x*0.5)
    * cos(t*WIND_GRASS_FREQ*0.7 + worldPos.z*0.3);
    d.x += gWave * WIND_GRASS_AMP * gW;

    // ■ LEAVES: Y-axis input = vertical phase variation ■■■■■■■■■■
    float lW = step(abs(entityId - 10002.0), 0.5);
    float lWave = sin(t*WIND_LEAVES_FREQ + worldPos.y*0.2)
    * sin(t*WIND_LEAVES_FREQ*1.3 + worldPos.x*0.15);
    d.xz += vec2(lWave) * WIND_LEAVES_AMP * lW * vec2(1.0, 0.7);

    // ■ CROPS: higher frequency = nervous sway ■■■■■■■■■■■■■■■■■■■■
    float cW = step(abs(entityId - 10003.0), 0.5);
    d.x += sin(t*WIND_CROPS_FREQ + worldPos.z) * WIND_CROPS_AMP * cW;

    // ■ RAIN: faster animation (forest shivering in storm) ■■■■■■
    // rainStrength: Iris built-in, 0.0=clear, 1.0=heavy
    d *= mix(1.0, RAIN_WIND_MULT, rainStrength);

    // ■ GROUND LOCK — THE MOST IMPORTANT PART ■■■■■■■■■■■■■■■■■■■■
    // Problem: without this, ALL vertices move = base floats.
    // In Minecraft UV space: V=0 is the TOP of a texture.
```

```
// For a 2-tall grass sprite, top vertex has vaUV0.y < 0.5.  
// step(0.5, vaUV0.y) = 1 for bottom vertex (V >= 0.5)  
// 1 - step(0.5, vaUV0.y) = 1 for TOP vertex only  
float isTop = 1.0 - step(0.5, vaUV0.y);  
d *= isTop; // bottom vertex gets zero displacement  
  
return d;  
}  
  
// Usage in gbuffers_terrain.vsh:  
// #include "/include/displacement.glsl"  
// vec3 disp = getWindDisplacement(worldPos, mc_Entity.x, frameTimeCounter);  
// gl_Position = projectionMatrix * (modelViewMatrix * vec4(vaPosition,1.0)  
// + vec4(disp, 0.0));
```

Error: Ghosting / floating grass

The isTop mask is missing or vaUV0.y check is inverted.

Debug: temporarily set WIND_GRASS_AMP to 2.0. Ghosting will be extreme and obvious.

Fix: ensure "float isTop = 1.0 - step(0.5, vaUV0.y);" and "d *= isTop;" are both present.

Verify the displacement is added BEFORE the gl_Position assignment, not after.

SECTION 4

Water — The Cinematic Glass

Gerstner physics, Beer-Lambert math, BSL clarity vs Spook dusk darkening

4.1 — Why Gerstner, Not 3D Noise

Most horror shaders use 3D Worley noise for water. It requires texture3D() calls — forbidden on the 1650 Ti (too slow, too much VRAM). Gerstner waves are pure trigonometry computed in the vertex shader. Zero VRAM cost. Physically accurate. Faster than any texture lookup. Use two layers at different directions and frequencies mixed at 0.5 to create organic, non-repeating interference patterns.

Gerstner physics: $\text{phase} = \text{dot}(\text{direction}, \text{position.xz}) * \text{frequency} + \text{time} * \text{speed}$. $\text{X-displacement} = \text{amplitude} * \text{dir.x} * \cos(\text{phase})$. $\text{Y-displacement} = \text{amplitude} * \sin(\text{phase})$. $\text{Z-displacement} = \text{amplitude} * \text{dir.z} * \cos(\text{phase})$. Unlike simple Y-only sine waves, Gerstner moves vertices in circular arcs — sharp peaked crests, flat troughs — exactly like ocean water.

```
// include/water.glsl

vec3 gerstnerWave(vec2 pos, float t, vec2 dir,
float freq, float amp, float spd) {
float ph = dot(dir, pos) * freq + t * spd;
return vec3(amp*dir.x*cos(ph), amp*sin(ph), amp*dir.y*cos(ph));
}

vec3 getWaterNormal(vec2 worldUV, float t) {
vec3 w1 = gerstnerWave(worldUV, t, normalize(vec2(1.0, 0.6)),
WATER_WAVE_FREQ, WATER_WAVE_AMP, 0.8);
vec3 w2 = gerstnerWave(worldUV*1.5, t, normalize(vec2(-0.7, 1.0)),
WATER_WAVE_FREQ*1.3, WATER_WAVE_AMP*0.6, 1.1);
vec3 c = mix(w1, w2, 0.5);
return normalize(vec3(c.x*2.0, 1.0, c.z*2.0));
}

// Beer-Lambert light absorption (deeper = more teal/green)
// At depth=1 block, exp(-0.10 * 8) = 0.45 — 55% blue absorbed
vec3 applyWaterAbsorption(vec3 color, float depth) {
vec3 ab = vec3(WATER_ABSORB_R, WATER_ABSORB_G, WATER_ABSORB_B);
return color * exp(-ab * depth * 8.0);
}

// Night/dusk darkening: water becomes teal-black mirror at midnight
// dayLight: normalized 0-1 sky brightness (from skyLight uniform)
vec3 applyWaterNightDarken(vec3 waterColor, float dayLight) {
```

```
float dark = 1.0 - dayLight * WATER_NIGHT_DARKEN;

// At midnight (dayLight~0): dark = 1.0 (full darken)
// At noon (dayLight~1.0): dark = 1 - 0.55 = 0.45 (barely affected)
// Shift toward teal-black: multiply green/blue slightly
return waterColor * vec3(dark * 0.85, dark, dark * 0.95);
}

// Chromatic aberration at screen edges
vec2 getChromaticAberration(vec2 uv) {
    vec2 c = uv - 0.5;
    return c * length(c) * WATER_CHROMA;
}
```

Day 3 Water Visual Check

Shallow water (< 1 block): retains original color, clear like BSL.

Deep water (> 3 blocks): visibly shifts to teal-green.

At midnight: water surface becomes near-black teal mirror with sharp moon reflection.

At screen edges: faint chromatic aberration split (sub-pixel RGB fringe).

Wave surface: two interference patterns visible, not a single repeating sine.

SECTION 5

Shadow, God Rays & Depth

Cosine distortion, PCF soft shadows, radial blur god rays, acne fix

5.1 — Shadow Map: Cosine Distortion Mathematics

Standard shadow maps distribute texels evenly across the shadow distance. At 128 blocks and 2048 resolution: $128/2048 = 0.0625$ blocks per texel. That is adequate close up but wastes resolution on distant shadows the player cannot see clearly anyway. Cosine distortion remaps the UV coordinates so the center (player position) gets more texels, the periphery gets fewer. The distortion factor 0.9 concentrates 90% of resolution in the inner radius. Result: near shadows are sharp and detailed; far shadows are softer but acceptable.

```
// include/shadow.glsl

// Distort shadow UVs toward center for better near-player resolution.
// Call this in shadow.vsh on the shadow clip position.
vec2 distortShadowUV(vec2 uv) {
    float d = length(uv);
    float dm = mix(d, 1.0, SHADOW_DISTORTION); // 0.9
    return uv / dm;
}

// Full shadow sampling with PCF 3x3 soft edges
float getShadow(vec3 worldPos, vec3 worldNormal, sampler2D shadowMap) {
    // Normal offset prevents self-shadowing acne
    vec3 offs = worldPos + worldNormal * SHADOW_NORMAL_OFFSET;
    vec4 shadowClip = shadowProjection * shadowModelView * vec4(offs, 1.0);
    vec3 sNDC = shadowClip.xyz / shadowClip.w;
    vec2 sUV = sNDC.xy * 0.5 + 0.5;
    float sDepth = sNDC.z * 0.5 + 0.5;

    // Apply distortion
    sUV = distortShadowUV(sUV * 2.0 - 1.0) * 0.5 + 0.5;

    // PCF 3x3 - Percentage Closer Filtering for soft shadow edges
    float shadow = 0.0;
    vec2 tSz = 1.0 / vec2(SHADOW_RESOLUTION);
    for (int x = -1; x <= 1; x++)
        for (int y = -1; y <= 1; y++)
            shadow += step(sDepth - 0.001,
                texture(shadowMap, sUV + vec2(x,y)*tSz).r);
    return shadow / 9.0; // normalize 3x3 grid
}
```



```
}

// Radial screen-space blur for god rays (NO volumetrics – too slow)
// sunScreenPos: sun projected to screen 0-1 space
vec3 getGodRays(sampler2D col, vec2 uv, vec2 sunPos) {
    vec2 dv = (uv - sunPos) / float(GODRAY_SAMPLES);
    float dec = 1.0;
    vec3 ray = vec3(0.0);
    vec2 suv = uv;
    for (int i = 0; i < GODRAY_SAMPLES; i++) {
        suv -= dv;
        vec3 s = texture(col, suv).rgb;
        float lum = dot(s, vec3(0.299, 0.587, 0.114));
        // Only bright sky pixels contribute – not terrain
        ray += s * dec * step(0.8, lum);
        dec *= GODRAY_DECAY;
    }
    // Gate on sun-above-horizon to prevent underground rays
    float sunUp = max(0.0, dot(normalize(sunPosition), vec3(0.0,1.0,0.0)));
    return ray * GODRAY_STRENGTH / float(GODRAY_SAMPLES) * sunUp;
}
```

Error: Shadow acne (dark striping on surfaces)

Increase SHADOW_NORMAL_OFFSET from 0.02 to 0.04 in settings.glsl.

Ryzen-specific: AMD FPU rounding may require 0.05.

If acne remains: reduce SHADOW_DISTANCE (smaller range = more depth precision).

Test: stand on a flat stone surface in direct sun. Zero striping = success.

SECTION 6

Atmosphere, Fog & Distant Horizons

Exponential fog math, purple sky derivation, DH seam hiding, rain desaturation

6.1 — Exponential Fog: The Math

Linear fog fades 0-1 over a fixed range. Exponential fog uses: $\text{fog} = 1 - \exp(-\text{density} * \max(\text{distance} - \text{fogStart}, 0))$. At density=0.0005 and distance=200 blocks: fog = $1 - \exp(-0.068) = 0.066$ — barely 7%. At 500 blocks: fog = $1 - \exp(-0.218) = 0.196$ — 20%. At 1000 blocks: fog = $1 - \exp(-0.468) = 0.374$ — 37%. This matches real atmospheric scattering and gives BlendHer the "Clarity vs Mystery" balance: the player's immediate world is razor sharp, the horizon fades into mystery.

```
// include/atmosphere.glsl

// Derive fog color from sky: desaturate -15%, add +5% purple
vec3 getFogColor(vec3 sky) {
    float luma = dot(sky, vec3(0.299, 0.587, 0.114));
    vec3 desat = mix(sky, vec3(luma), FOG_SAT_REDUCE);
    // Purple shift: boost red slightly, boost blue more
    return desat + vec3(FOG_PURPLE_SHIFT*0.5, 0.0, FOG_PURPLE_SHIFT);
}

float getFogDensity(float dist) {
    // Exponential from FOG_START (64 blocks)
    float local = 1.0 - exp(-FOG_DENSITY * max(dist - FOG_START, 0.0));

    // DH fog wall: smoothstep 1200->1500 blocks rises to 0.9 opacity
    // This hides the LOD seam where Distant Horizons chunks begin
    float dh = smoothstep(FOG_DH_START, FOG_DH_END, dist) * 0.9;

    return max(local, dh); // DH wall always wins past 1500 units
}

vec3 applyFog(vec3 scene, vec3 sky, float dist) {
    vec3 fc = getFogColor(sky);
    float ff = getFogDensity(dist);

    // RAIN: desaturate scene — do NOT add dense fog (kills visibility)
    // Target: player keeps 32-block visibility even in heavy rain
    float luma = dot(scene, vec3(0.299, 0.587, 0.114));
    float desat = rainStrength * RAIN_DESAT; // 0.75 max
    scene = mix(scene, vec3(luma), desat);

    return mix(scene, fc, ff);
}
```

```
}
```

DH Integration: shaders.properties Required

Add this exact line to shaders.properties: iris.features=DISTANT_HORIZONS

Without it, Iris does not expose DH depth buffers to your composite pass.

After adding: restart Minecraft completely (not just F3+R reload shaders).

Verify FOG_DH_END matches your DH render distance setting in blocks.

Example: DH at 64 chunks = 1024 blocks. Set FOG_DH_START=900, FOG_DH_END=1100.

SECTION 7

THE BLENDHER VIBE — Complete Aesthetic Bible

The five visual states. The color targets. The purple thread. The shipping checklist.

"BlendHer is a high-performance psychological experience. It is the Golden Hour of BSL meeting the Dead of Night of Spooklementary, held together by a thin, glowing purple thread."

7.1 — The Five Visual States

Every shader decision you make must be evaluated against one of these five states. If a change improves State 1 but breaks State 4, it is rejected. All five states must pass simultaneously.

State 1: High-Noon Vibrancy (The BSL Heritage)

Birch forest, noon, direct sunlight. This is where the BSL mode dominates completely. The world must feel premium, safe, and alive.

- **Sunlight:** Heavy, golden nectar. Not just white light — a warm saturating amber-gold (BSL_WARMTH vec3: 1.05, 0.98, 0.88).
- **Grass:** Punchy emerald green — NOT olive. Saturation must be at maximum in direct light.
- **Leaves:** Vibrant filters for sunlight. Subsurface scattering simulation: leaves facing sun glow brighter.
- **Water:** Crystal clear with BSL-style sharp specular reflections. Sky reflected sharply. WATER_REFLECT = 0.8.
- **Clouds:** Thick, painterly, BSL-style. Not flat — they have mass and volume.
- **Feeling:** Total safety. Every detail visible. Zero purple contamination in this state.

GLSL target: modeBlend = 1.0 (full BSL). totalLight > 0.15.

State 2: The Creeping Shadows (Spook Injection)

Step under dense oak canopy. Direct sunlight is gone. The Blend begins. This is the emotional heart of BlendHer — the transition must feel unmistakable but not aggressive. The player must sense something has changed.

- **Saturation crush:** Vibrant greens immediately pull back into deep olive-grey. SPOOK_DESAT = 0.55.
- **The purple floor:** Look into any deep shadow corner. It is NOT black. It is menacing charcoal-purple. This is the BlendHer signature — darkness with a pulse.
- **Torch sanctuary:** A single torch in this darkness becomes a circle of warm safety. Bloom threshold 1.2 — torches glow without overexposure.
- **Contrast:** The edge between torchlight and shadow is sharp and dramatic — BSL gold vs Spook purple in the same frame.

GLSL target: modeBlend = 0.0-0.3. totalLight 0.02-0.08. ambiFloor active.

State 3: The Purple Haze Sunset

The sky transitions from BSL orange to haunting violet-purple as the sun dips. This is where BlendHer finds its true identity — the moment the world stops feeling safe.

- **Sky gradient:** Horizon bleeds from amber-orange to violet-purple. The fog color inherits this gradient with -15% saturation shift.
- **Atmosphere:** Exponential fog rises visibly. DH fog wall creates soft purple veil at the horizon. World feels infinite but claustrophobic.
- **Water:** Clear ocean turns into dark teal-black mirror. Moon reflection sharp and sparkling. Water beneath: cold, bottomless, dangerous.
- **God rays:** Sunlight shafts through trees as the sun drops — radial SS blur with 0.4 strength and 0.96 decay.

GLSL target: worldTime 11000-13000. modeBlend transitions 1.0->0.2. Fog density rising.

State 4: Playable Horror — Midnight

Most horror shaders fail here: they blind the player. BlendHer does the opposite. At midnight, the player must be EMPOWERED to navigate while remaining ANXIOUS. This is the hardest balance to achieve and the most important to get right.

- **Moonlight cyan:** Terrain is lit by cool cyan-white moonlight (MOON_CYAN_R/G/B = 0.72, 0.88, 1.00). Silhouettes are defined. The player can see paths.
- **Shadow anxiety:** While paths are clear, shadow interiors are impenetrable. The purple ambient floor keeps the player's brain second-guessing every dark corner.
- **Water at night:** Teal-black mirror surface. Moon reflection is sharp and sparkling. Looks cold and bottomless. WATER_NIGHT_DARKEN = 0.55.
- **NO blinding:** At no point should the player be unable to navigate. Visibility target: 32 blocks minimum on moonlit paths.

GLSL target: worldTime 13000-23000. isNight = 1.0. moonTint active at 0.18 intensity.

State 5: The Storm — Rain Mode

When it rains, BlendHer strips vibrancy entirely. The world goes near-grayscale. Only the purple haze and the shimmer of wet blocks remain. The forest shudders.

- **Desaturation:** rainStrength drives RAIN_DESAT = 0.75. The world becomes grey-purple. Not grey-brown — the purple ambient floor survives the rain.
- **Wind:** Foliage animation speed multiplies by RAIN_WIND_MULT = 1.8. The forest visibly shivering.
- **NO dense fog:** Rain desaturates — it does NOT add thick fog. Player visibility stays >= 32 blocks. This is the key distinction from most rain shaders.
- **Wet shimmer:** Rain wetness increases specular reflection on terrain. Wet blocks reflect moonlight/torchlight more sharply.

GLSL target: rainStrength = 1.0. desaturation at max. Wind amplitude *= 1.8.

7.2 — The Purple Thread Principle

The "thin glowing purple thread" that holds BlendHer together is the ambient floor. It appears in EVERY dark area, in EVERY weather state, at EVERY time of day. It is the signature. Without it, BlendHer becomes just another high-performance shader. With it, the world feels like it has a hidden pulse — something alive beneath the surface. Never turn it off. Never reduce AMBIENT_B below 0.07. Never make AMBIENT_THRESHOLD above 0.08.

7.3 — Color Targets Per State (Reference Table)

Visual State	modeBlend	Purple Floor	Sky Feel	Water
Noon vibrancy	1.0 (full BSL)	Invisible	Bright blue, thick clouds	Crystal clear, sharp reflect
Canopy shadows	0.0-0.3	Active — pulse visible	Filtered gold-green	Dark teal at depth
Sunset transition	0.2-0.8	Growing	Orange to violet gradient	Deepening teal-black
Midnight	0.1 (Spook + cyan)	Dominant	Deep indigo-purple	Teal-black mirror, moon spark
Rain storm	0.0 (Spook max)	Survives rain	Grey-purple wash	Wet shimmer, dark

7.4 — The Final Shipping Checklist

Run through every item on this checklist before declaring the shader complete. All items must pass. There is no partial credit.

Category	Pass Criterion
FPS	60 FPS minimum in dense oak forest at noon (shadows + bloom active on GTX 1650 Ti)
FPS	90 FPS target on open plains with clear sky
Acne	Zero shadow striping on any flat stone, wood, or dirt surface in direct sun
Grass	Grass base stays ground-locked — zero floating when wind is active
Purple	Underground cave with zero light sources shows charcoal-purple tint — NOT black void
Sun	Direct sunlight at noon: zero purple tint. Pure BSL warmth.
Torch	Single torch in a dark cave: warm golden sanctuary. Bloom visible, not overexposed.
Water	Deep water (3+ blocks): visibly teal. Shallow: clear. Night: teal-black mirror.
DH	Fly to DH LOD boundary: transition invisible. Fog wall hides seam completely.
Rain	Heavy rain: world desaturates to grey-purple. Player sees 32+ blocks. No fog blindness.
Midnight	Navigate terrain by moonlight only: paths visible via cyan tint. Shadows remain anxious.
Sunset	Sky gradient transitions from amber to violet-purple with visible atmospheric fog rise.
God rays	God rays visible when sun is above horizon through foliage. Absent underground.
Errors	latest.log contains zero shader compilation ERRORS.
VRAM	GPU-Z shows shader VRAM usage under 200 MB (not counting world chunks).
Includes	Zero duplicate functions across .fsh/.vsh files — all logic in includes.

"It is clear. It is vibrant. It is purple. It is BlendHer."